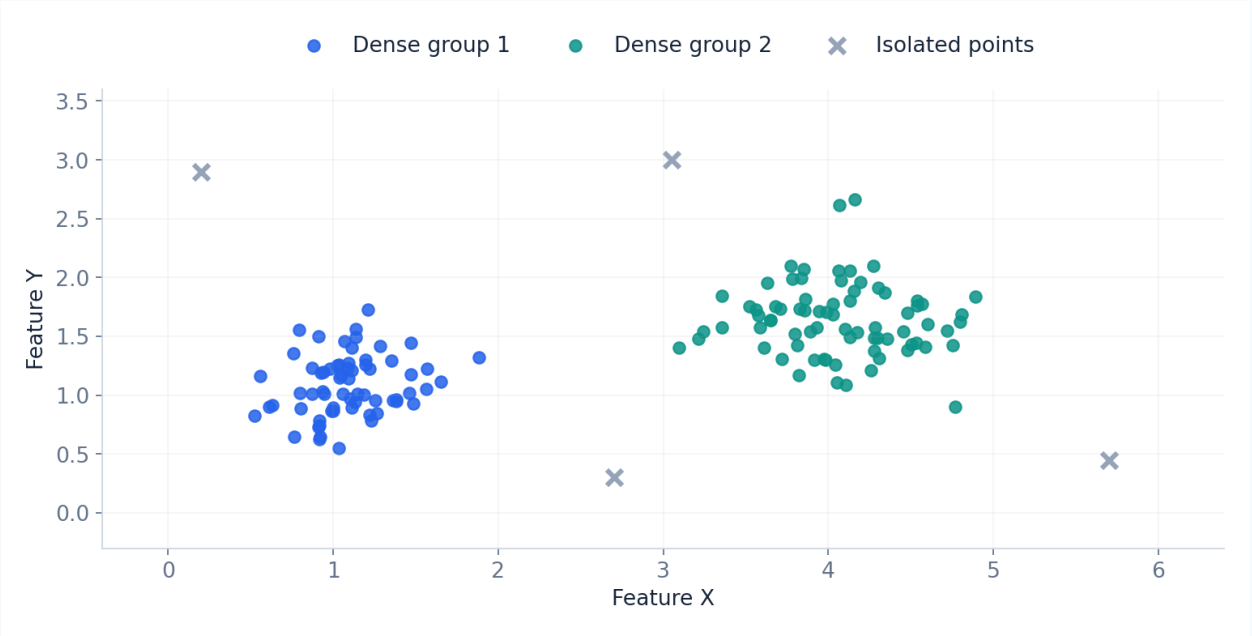
UNSUPERVISED LEARNING

DBSCAN

Density-Based Clustering

Step-by-step concepts, calculations, illustrations, and scikit-learn implementation



Focus: core • border • noise • cluster growth

LEARNING PATH

What will we learn today?

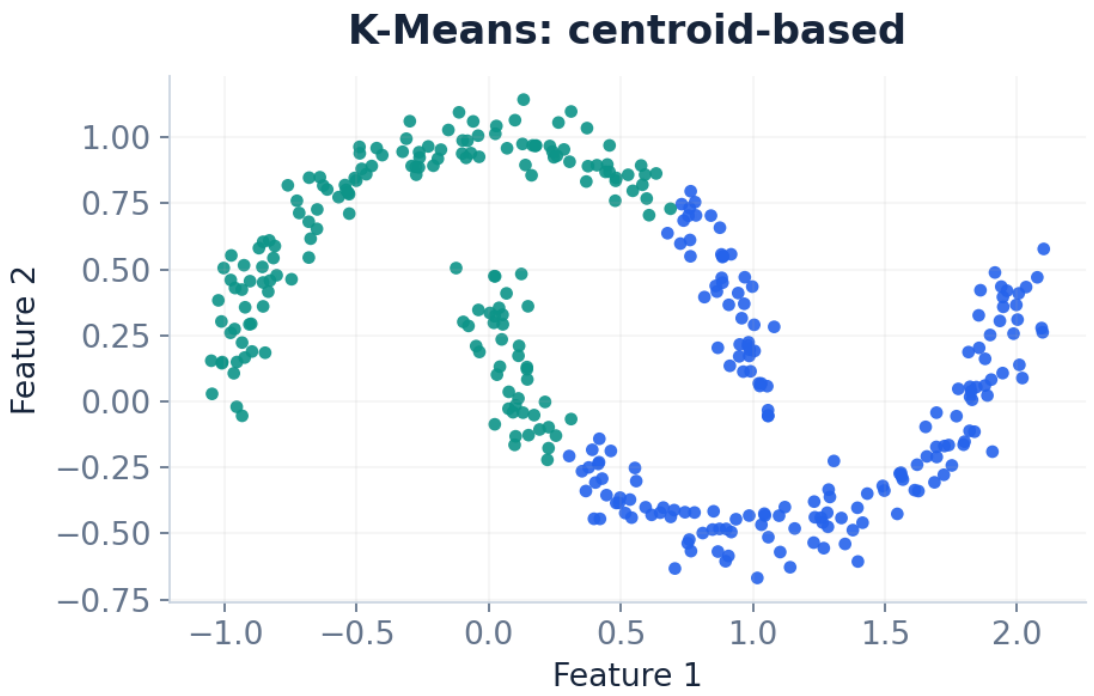
A continuous flow from the problem to a working DBSCAN model.

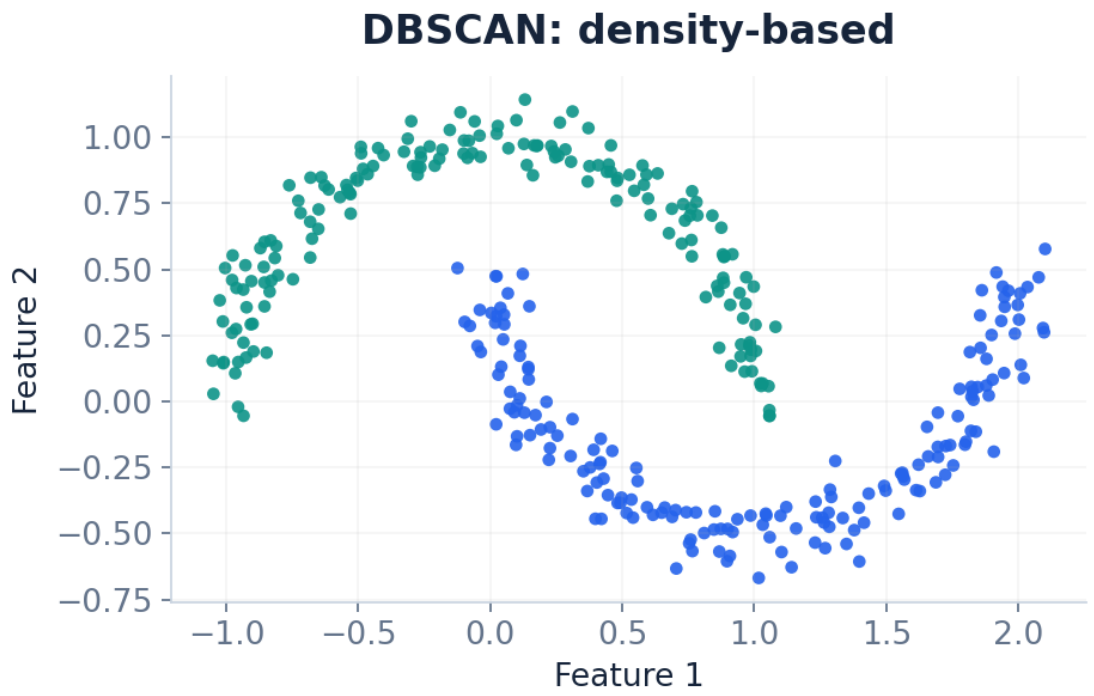
01	**The need**	Why K-Means is not always enough
02	**Dense neighborhoods**	Understand eps and min_samples
03	**Types of points**	Core, border, and noise
04	**Worked example**	Calculate distances and grow a cluster
05	**Use in Python**	Tune eps, scale features, plot results

PROBLEM

Why do we need DBSCAN?

K-Means may split curved clusters because it relies on distance to centroids.





KEY TAKEAWAY **DBSCAN can follow irregular cluster shapes and identify noise without setting K.**

The central idea: find dense regions

A dense region contains enough nearby points; isolated points may be noise.



Find neighbors

Look inside a fixed distance radius.

Check density

Count how many points are nearby.

Grow clusters

Connect nearby core points.

KEY TAKEAWAY **No centroid is calculated or updated.**

DBSCAN uses two main parameters

Both parameters determine whether a point belongs to a dense neighborhood.

eps (epsilon)

Maximum distance between a point and its neighbors. Think: size of the neighborhood radius.

min_samples

Minimum number of points within eps required for a core point. The point itself is included.

For our example

`eps = 1.5`

`min_samples = 4`

KEY TAKEAWAY **A core point has at least 4 points inside its eps neighborhood—including itself.**

Three possible types of points

The classification of a point is determined by neighborhood counts and reachable core points.

CORE POINT

At least min_samples points (including itself) are within eps. Core points can expand a cluster.

Core = cluster expansion

BORDER POINT

Does not have enough neighbors to be core, but lies within eps of a core point.

Border = cluster member

NOISE POINT

Is neither core nor border. It is not assigned to a cluster.

Noise label = -1

Use one six-point dataset throughout

The same six observations will be used for every distance and cluster calculation.

Point	X	Y
A	0	0
B	0	1
C	1	0
D	1	1
E	2	0
F	4	1



eps = 1.5

min_samples = 4

Step 1 — Calculate distances from A

DBSCAN checks whether each distance is less than or equal to eps = 1.5.

Euclidean distance

$$d = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

Example: distance A → D

$$\sqrt{[(1-0)^2 + (1-0)^2]} = \sqrt{2} = 1.414$$

From A to	Distance	In eps?
A	0.000	Yes
B	1.000	Yes
C	1.000	Yes
D	1.414	Yes
E	2.000	No
F	4.123	No

KEY TAKEAWAY **A has 4 points in its neighborhood: A, B, C, D. Therefore A is a core point.**

Step 2 — Classify every point

Count the samples inside eps = 1.5, including the point under inspection.

Point	Neighbor count	Neighbor IDs	Point type
A	4	A, B, C, D	CORE
B	4	A, B, C, D	CORE
C	5	A, B, C, D, E	CORE
D	5	A, B, C, D, E	CORE
E	3	C, D, E	BORDER
F	1	F	NOISE

4

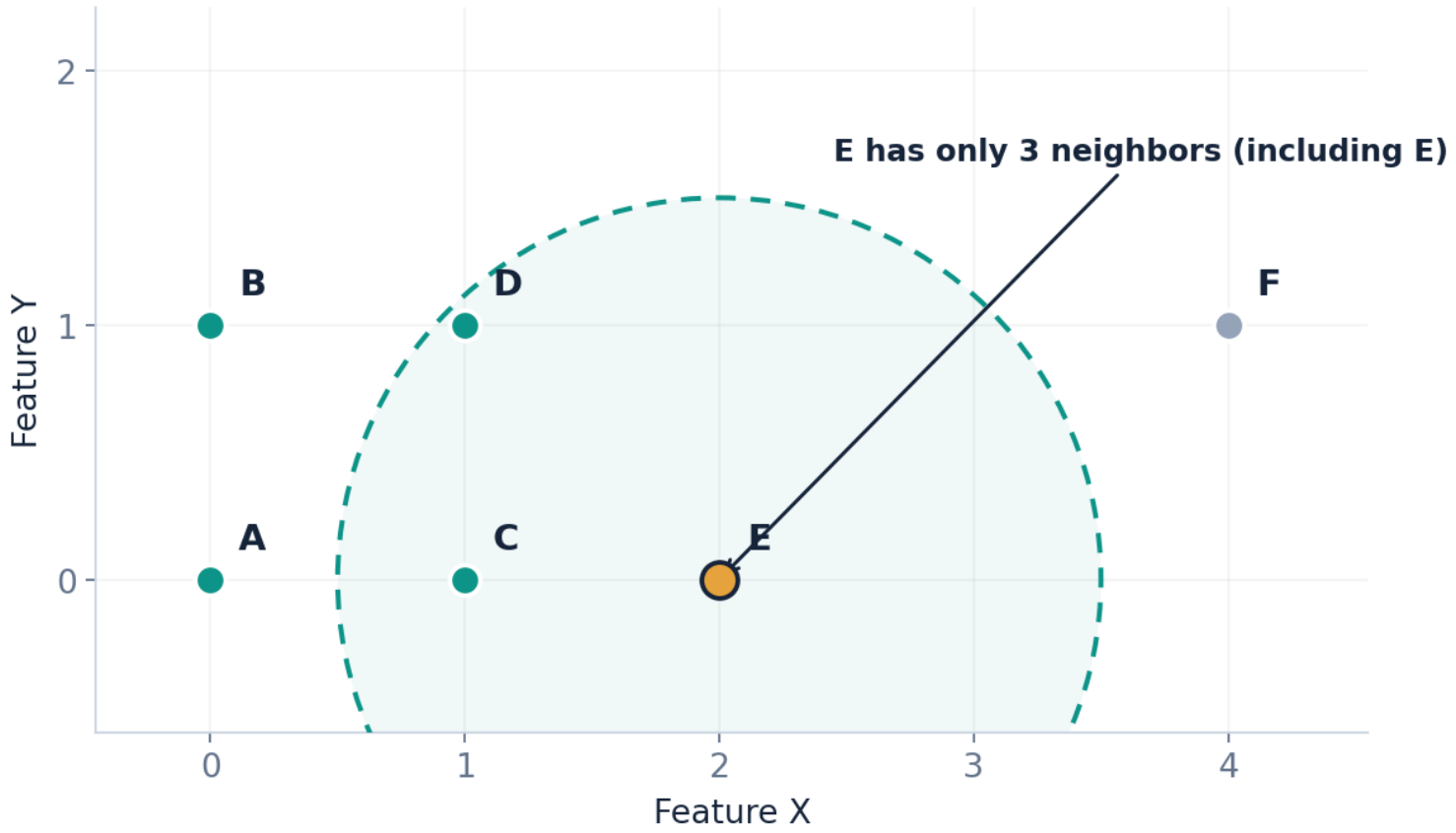
minimum count for a core point

KEY TAKEAWAY **Neighborhood count alone determines core status; border status also depends on nearby core points.**

WORKED EXAMPLE

Why is E a border point?

E is close enough to core points C and D, but its own neighborhood is too small.



E has 3 neighbors

C, D and E

3 is less than 4

E is not a core point.

Within eps of C & D

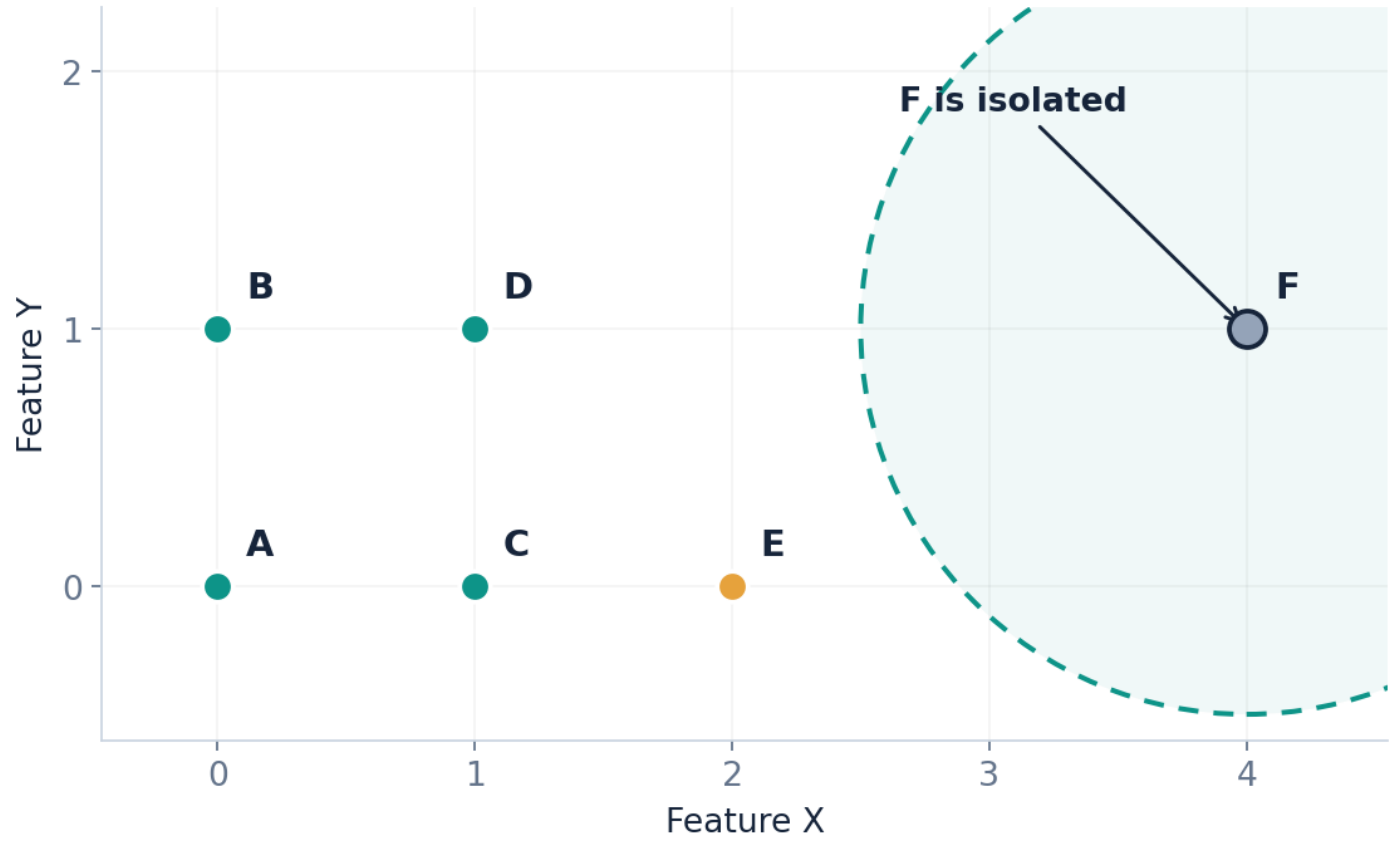
E joins their cluster.

KEY TAKEAWAY **A border point belongs to a cluster but cannot expand it further.**

WORKED EXAMPLE

Why is F a noise point?

F has no other sample within eps = 1.5, and no core point can reach it.



F has 1 neighbor

Only itself is inside its eps radius.

No reachable core

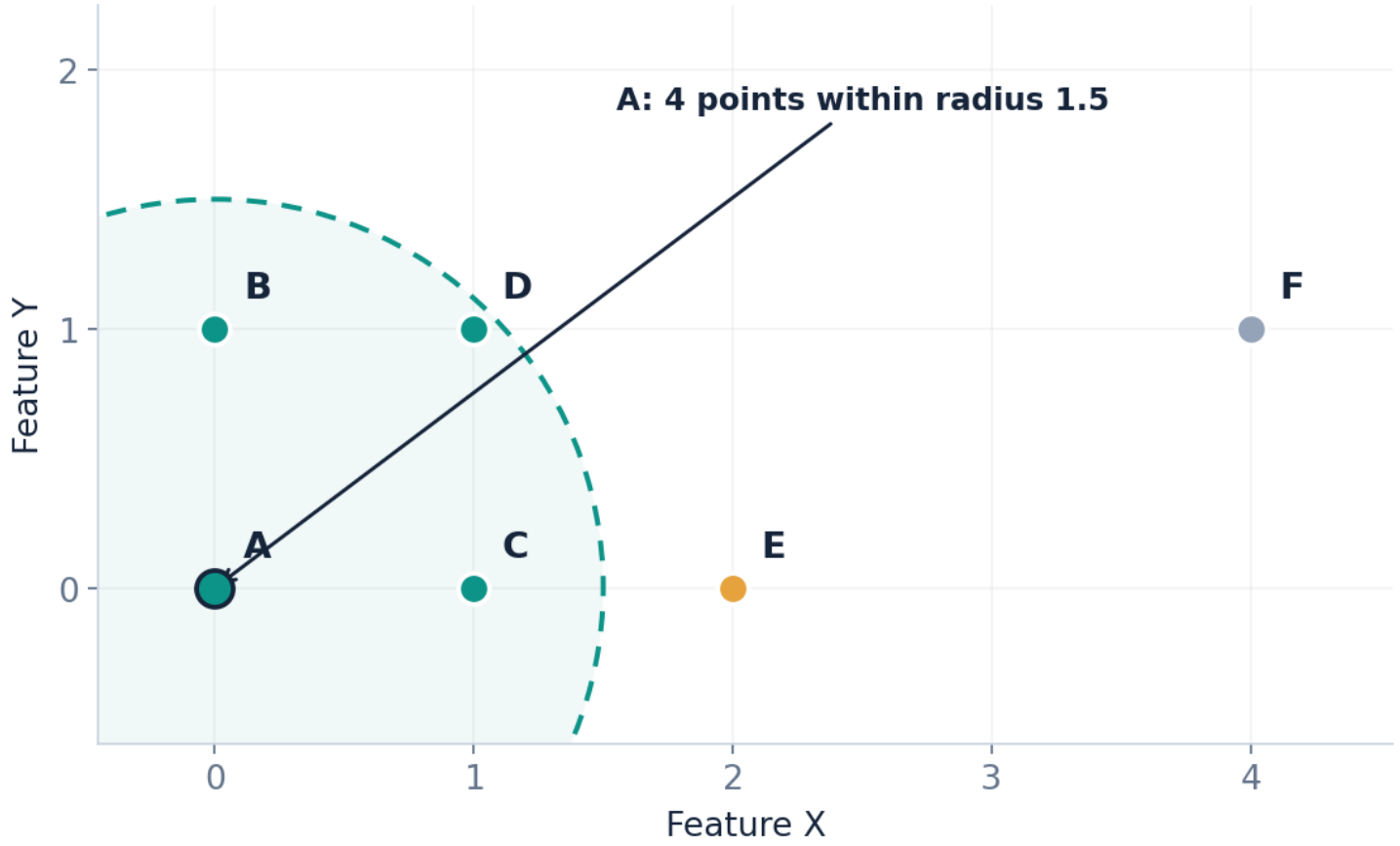
F is not a border point.

F → label −1

KEY TAKEAWAY **Noise is not counted as a normal cluster in scikit-learn.**

Step 3 — Start a cluster from core point A

A directly reaches B, C and D within eps = 1.5.



A is core

The neighborhood count is 4.

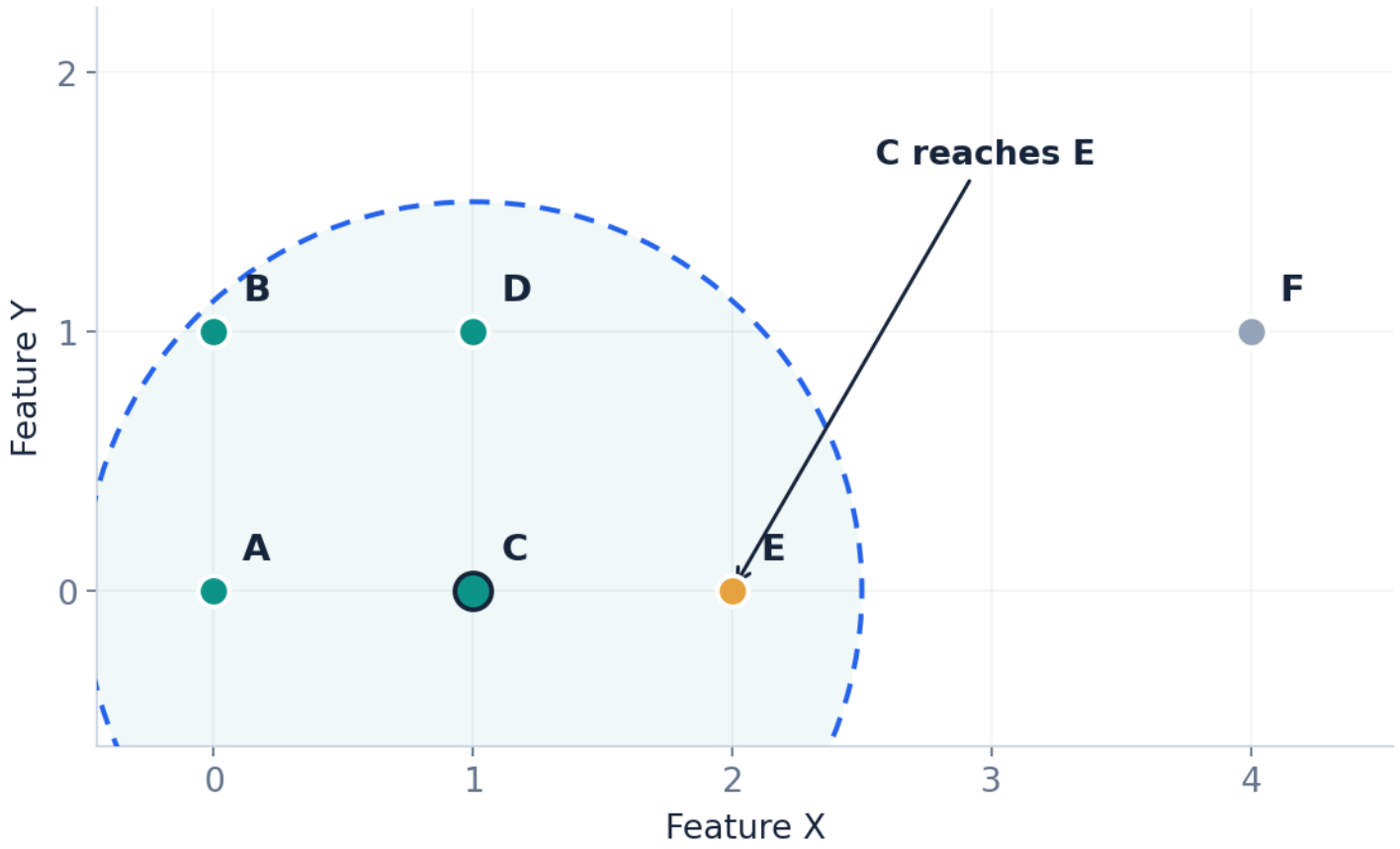
Start Cluster 0

Initially assign A, B, C, D.

Cluster 0: A, B, C, D

Step 4 — Expand through other core points

C is core; its neighborhood includes E, so the cluster grows beyond A's radius.



Inspect C

C is already part of Cluster 0.

Find new neighbor E

E is reachable from core point C.

Stop at E

E is border, so it does not expand.

KEY TAKEAWAY **Final result: Cluster 0 = {A, B, C, D, E}; Noise = {F}.**

DBSCAN algorithm: complete process

Repeat neighborhood checks until all points have been visited.

1. **Select an unvisited point**
2. **Find points within eps**
3. **Is neighborhood count ≥ min_samples?**

NO

Temporarily mark noise

May later become a border point.

YES

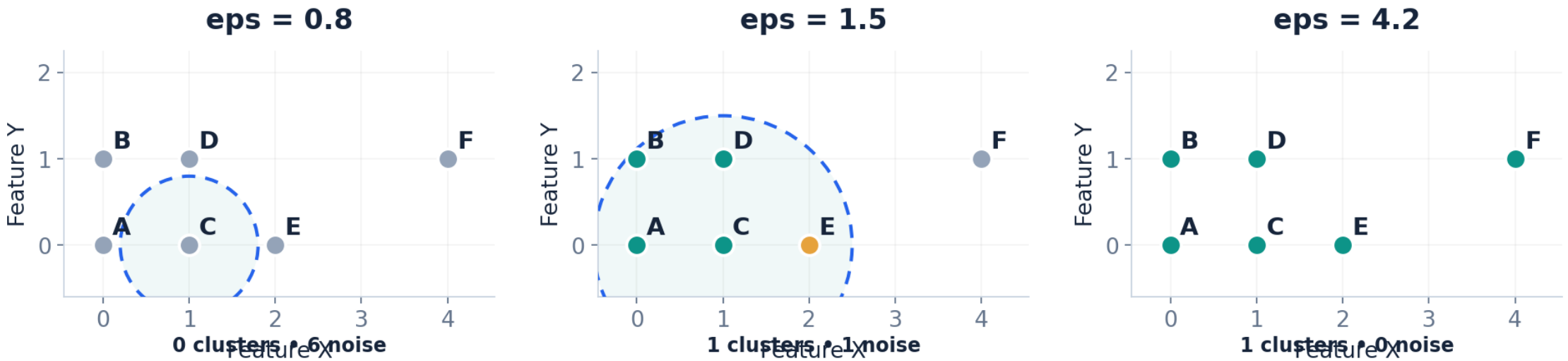
Start / extend a cluster

Expand through reachable core points.

Continue until every point is visited

What changes when eps increases?

Keep min_samples = 4 and change only the neighborhood radius.



Small eps: more noise • **Moderate eps: useful separation** • **Large eps: groups may merge**

What changes when min_samples increases?

Keep eps = 1.5; make the core-point density requirement stricter.

min_samples	Core points	Border points	Noise
3	A, B, C, D, E	None	F
4	A, B, C, D	E	F
5	C, D	A, B, E	F
6	None	None	A, B, C, D, E, F

Lower min_samples

Easier to qualify as core.

Higher min_samples

Requires a denser neighborhood.

Scaling changes what “nearby” means

Distance-based algorithms can be dominated by features with large numerical scales.

Feature	Original values	Typical magnitude
Age	18 to 60	Tens
Annual income	₹2,00,000 to ₹20,00,000	Hundreds of thousands

Recommendation

Scale numerical features (for example, StandardScaler) before selecting eps. After scaling, tune eps again.

```
X_scaled = StandardScaler().fit_transform(X)
```

KEY TAKEAWAY **eps is expressed in the units of the feature space used for fitting.**

DBSCAN with scikit-learn — six-point example

Fit the algorithm and inspect the cluster labels and core-sample indices.

```
import numpy as np
from sklearn.cluster import DBSCAN

X = np.array([[0, 0], [0, 1], [1, 0],
              [1, 1], [2, 0], [4, 1]])

model = DBSCAN(eps=1.5, min_samples=4)
labels = model.fit_predict(X)

print(labels)                  # [0 0 0 0 0 -1]
print(model.core_sample_indices_)  # [0 1 2 3]
```

labels_

0 = cluster ID; −1 = noise.

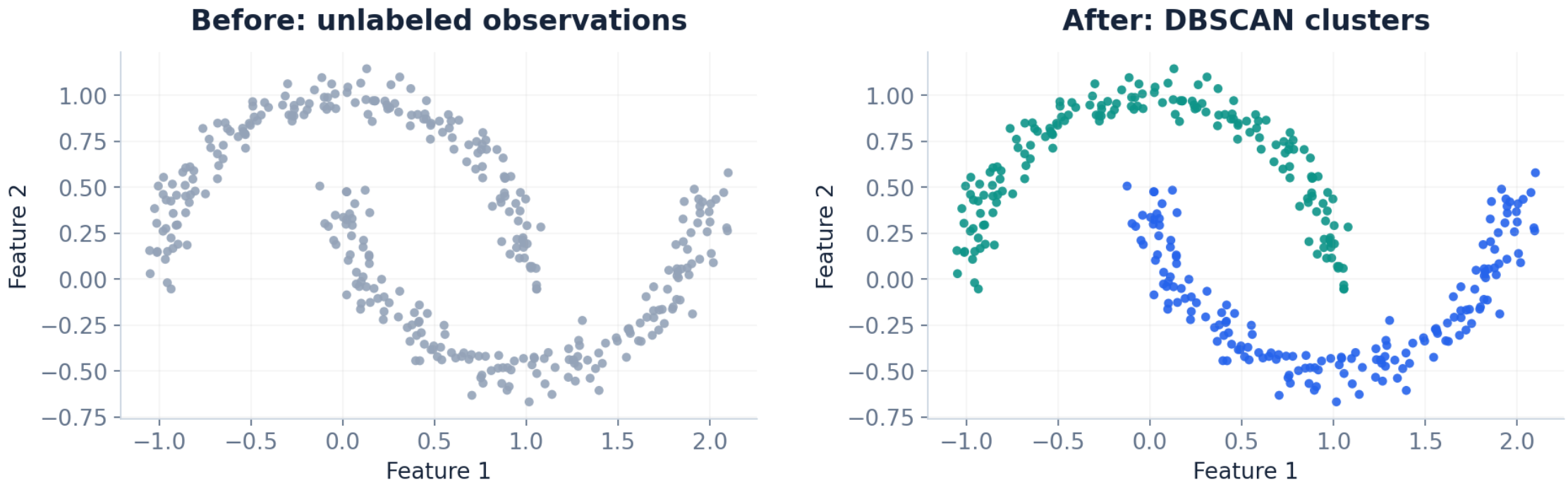
core_sample_indices_

Positions of the core points.

No train/test split is needed to discover clusters in this demonstration.

DBSCAN can identify curved clusters

A synthetic two-moons dataset illustrates density-based clustering on non-circular shapes.



KEY TAKEAWAY **DBSCAN discovered the two curved groups without specifying K = 2.**

Complete example: two-moons clustering

A short experiment students can run and visualize in a notebook.

```
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

X, _ = make_moons(n_samples=350, noise=0.065, random_state=22)
labels = DBSCAN(eps=0.18, min_samples=5).fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis", s=25)
plt.title("DBSCAN on two moons")
plt.show()
```

For a real dataset, inspect feature scales and tune eps for that representation.

COMPARISON

K-Means vs DBSCAN

They solve similar clustering problems using different assumptions.

Dimension	K-Means	DBSCAN
Main principle	Distance to centroid	Density / reachability
Number of clusters	Specify K upfront	Not specified upfront
Cluster shapes	Often compact / convex	Can be irregular
Outliers	Every point assigned	Can label noise as −1
Key hyperparameters	n_clusters	eps, min_samples
New point labels	predict() available	No native predict()

KEY TAKEAWAY **Choose the method based on data geometry, density variation, and modeling needs.**

When does DBSCAN struggle?

Understand the limitations before using DBSCAN on a real dataset.

Different cluster densities

One eps may not suit both sparse and very dense regions.

High-dimensional features

Distances may become less informative; feature reduction can help.

Parameter sensitivity

eps that is too small or large changes cluster structure substantially.

Large datasets / deployment

Memory use can grow; scikit-learn DBSCAN has no native predict for unseen rows.

KEY TAKEAWAY **Always inspect the plot, noise percentage, and parameter stability.**

CLASS ACTIVITY

Check your understanding

Use eps = 1.5 and min_samples = 4 for the six-point dataset.

Q1 Why is point A called a core point?

Q2 Why does E join the cluster even though its count is only 3?

Q3 If eps = 0.8, how many clusters will DBSCAN find?

Q4 Does DBSCAN calculate centroids to grow clusters?

Answers: Q1 — count 4; Q2 — within eps of core C/D; Q3 — 0; Q4 — No.

DBSCAN: five things to remember

Use these points as a summary before moving to the Python notebook.

1. No need to predefine the number of clusters.
2. eps controls the neighborhood radius.
3. min_samples includes the point itself.
4. **Core points grow clusters; border points join; noise is −1.**
5. Scale features, inspect the k-distance graph, and tune eps.

Reference: scikit-learn DBSCAN documentation https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html